

Программирование на С

Лекция 5

Массивы

Зачем нужны массивы?

Допустим, нужно сохранить оценки пяти студентов.

```
int grade1 = 5;  
int grade2 = 4;  
int grade3 = 5;  
int grade4 = 3;  
int grade5 = 5;
```

А если студентов сто?

Много одинаковых переменных

```
int grade1;  
int grade2;  
int grade3;  
...  
int grade100;
```

Такой код неудобно писать и сложно использовать.

Что такое массив?

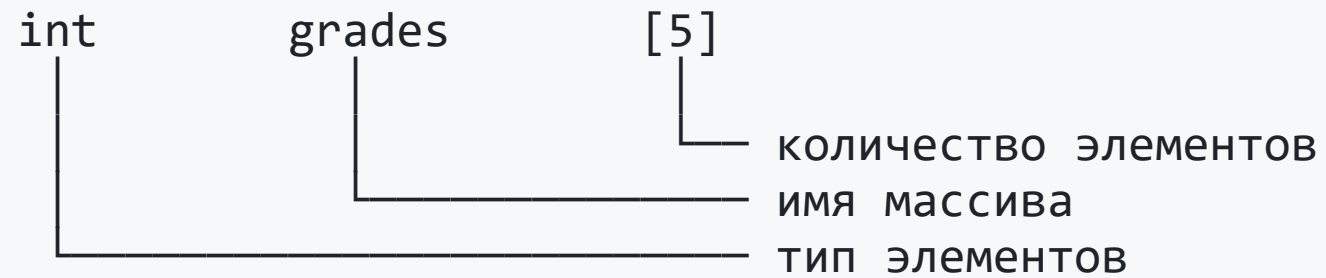
Вместо множества отдельных переменных можно использовать массив.

```
int grades[5];
```

Массив — это последовательность элементов одного типа, объединённых одним именем.

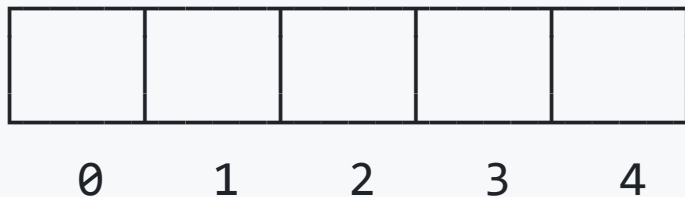
Объявление массива

```
int grades[5];
```



Элементы массива

Каждый элемент имеет свой номер.



Этот номер называется **индексом**.

Запись значения

```
grades[0] = 5;  
grades[1] = 4;  
grades[2] = 5;  
grades[3] = 3;  
grades[4] = 5;
```

Чтение значения

```
printf("%d\n", grades[0]);  
printf("%d\n", grades[3]);
```

Элемент массива — это обычная переменная

```
grades[0] = 5;  
grades[0] = grades[0] + 1;  
printf("%d\n", grades[0]);
```

С элементами массива можно делать всё то же самое, что и с обычными переменными.

Индексация начинается с нуля

В языке **C** первый элемент массива имеет индекс **0**.

Количество элементов: 5

Индексы:

0 1 2 3 4

Последний индекс всегда на единицу меньше количества элементов.

Инициализация массива

```
int grades[5] = {5, 4, 5, 3, 5};
```

После создания массива все значения будут записаны автоматически.

Используем цикл

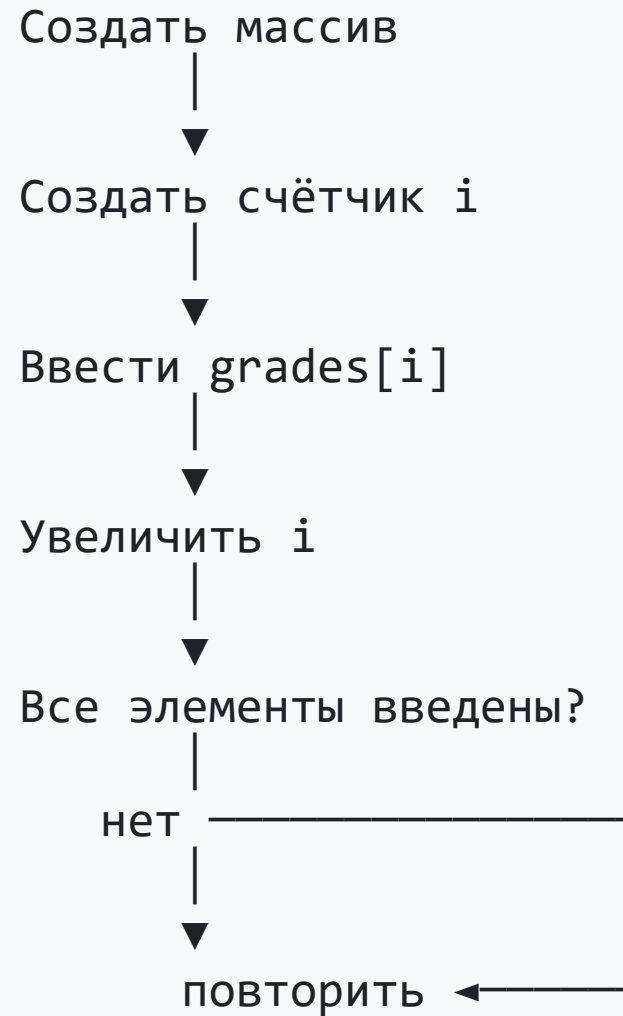
Теперь цикл становится гораздо полезнее.

```
int grades[5];  
  
for (int i = 0; i < 5; i++)  
{  
    scanf("%d", &grades[i]);  
}
```

Вывод массива

```
for (int i = 0; i < 5; i++)  
{  
    printf("%d\n", grades[i]);  
}
```

Что происходит?



Пример программы

```
#include <stdio.h>

int main()
{
    int grades[5];

    for (int i = 0; i < 5; i++)
    {
        scanf("%d", &grades[i]);
    }

    for (int i = 0; i < 5; i++)
    {
        printf("%d\n", grades[i]);
    }

    return 0;
}
```

Ввод

5
4
5
3
5

Вывод

5
4
5
3
5

Типичная ошибка

```
int grades[5];
```

Допустимые индексы:

```
0 1 2 3 4
```

Ошибка:

```
grades[5] = 10;
```

Нельзя обращаться к элементам за пределами массива.

Индексом может быть не только число

До сих пор мы обращались к элементам массива так:

```
arr[0]  
arr[1]  
arr[2]
```

Но внутри квадратных скобок может находиться и выражение.

Выражение в качестве индекса

```
int a = 5;  
int b = 2;  
  
int arr[10] = {0};  
  
arr[a + b] = 15;
```

Сначала вычисляется выражение:

$$a + b = 5 + 2 = 7$$

Поэтому команда

```
arr[a + b] = 15;
```

работает так же, как

```
arr[7] = 15;
```


Что произошло с массивом?

```
arr[a + b] = 15;
```

```
a + b = 7
```

Индекс	0	1	2	3	4	5	6	7	8	9
Значение	0	0	0	0	0	0	0	15	0	0

Значение 15 записано в элемент с индексом 7.

Чтение элемента

```
printf("%d\n", arr[a + b]);
```

Сначала снова вычисляется выражение:

$$a + b = 7$$

Затем программа читает:

```
arr[7]
```

Вывод

```
15
```

Полная программа

```
#include <stdio.h>

int main()
{
    int a = 5;
    int b = 2;

    int arr[10] = {0};

    arr[a + b] = 15;

    printf("%d\n", arr[a + b]);

    return 0;
}
```

Вывод

15

Что может быть индексом?

Число:

```
arr[3]
```

Переменная:

```
arr[i]
```

Арифметическое выражение:

```
arr[i + 1]  
arr[a + b]
```

Индексом массива может быть выражение, результатом которого является целое число.

Нужно следить за границами

```
int arr[10];
```

Допустимые индексы:

```
0 1 2 3 4 5 6 7 8 9
```

Поэтому значение выражения внутри скобок должно находиться в этом диапазоне.

```
arr[a + b]
```

Важно не только то, как записан индекс, но и какое значение получится после вычисления выражения.

Пример ошибки

```
int a = 7;  
int b = 5;  
  
int arr[10];  
  
arr[a + b] = 15;
```

```
a + b == 12
```

Но элемента `arr[12]` в массиве нет.

Любое выражение в качестве индекса допустимо только тогда, когда его значение не выходит за границы массива.

Основная идея

`arr[выражение]`



вычислить значение выражения



использовать результат как индекс

Например:

`arr[a + b]`

`arr[5 + 2]`



`arr[7]`

Пример программы

Пусть известны температуры воздуха за семь дней.

```
#include <stdio.h>

int main()
{
    int temperature[7] = {25, 27, 24, 23, 26, 28, 29};

    int day;

    printf("Введите номер дня: ");
    scanf("%d", &day);

    printf("Температура: %d\n", temperature[day]);

    return 0;
}
```

Ввод

3

Вывод

Температура: 23

**Предполагается,
что пользователь
вводит число от
0 до 6.**

Как работает программа?

temperature

Индекс	0	1	2	3	4	5	6
Значение	25	27	24	23	26	28	29

Пользователь вводит

```
day = 3
```

Программа выполняет

```
temperature[day]
```

То есть

```
temperature[3]
```

И получает значение

```
23
```

Пример программы

```
#include <stdio.h>

int main()
{
    int temperature[7];

    for (int day = 0; day < 7; day++)
    {
        scanf("%d", &temperature[day]);
    }

    for (int day = 0; day < 7; day++)
    {
        printf("%d\n", temperature[day]);
    }

    return 0;
}
```

Инициализация массива

Как и обычные переменные, массив можно инициализировать при создании.

```
int numbers[5] = {10, 20, 30, 40, 50};
```

После выполнения программы массив будет содержать:

Индекс	0	1	2	3	4
Значение	10	20	30	40	50

Размер можно не указывать

Если элементы перечислены полностью, компилятор сам определит размер массива.

```
int numbers[] = {10, 20, 30, 40, 50};
```

Это полностью эквивалентно записи

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Количество элементов определяется по списку инициализации.

Инициализация нулями

Если нужно создать массив, заполненный нулями, достаточно написать

```
int numbers[5] = {0};
```

Результат:

Индекс	0	1	2	3	4
Значение	0	0	0	0	0

Все элементы массива будут иметь значение 0 .

Можно указать не все элементы

```
int numbers[5] = {10, 20};
```

После инициализации массив будет иметь вид:

Индекс	0	1	2	3	4
Значение	10	20	0	0	0

Если список инициализации короче размера массива, остальные элементы автоматически заполняются нулями.

Нельзя указать слишком много элементов

```
int numbers[5] = {10, 20, 30, 40, 50, 60};
```

Размер массива равен 5, а в списке инициализации указано 6 элементов.

Компилятор сообщит об ошибке.

Способы инициализации массива

```
int a[5] = {10, 20, 30, 40, 50};
```

Все элементы заданы явно.

```
int a[] = {10, 20, 30, 40, 50};
```

Размер определяется автоматически.

```
int a[5] = {0};
```

Все элементы равны нулю.

```
int a[5] = {10, 20};
```

Остальные элементы автоматически становятся равны нулю.